

System Description

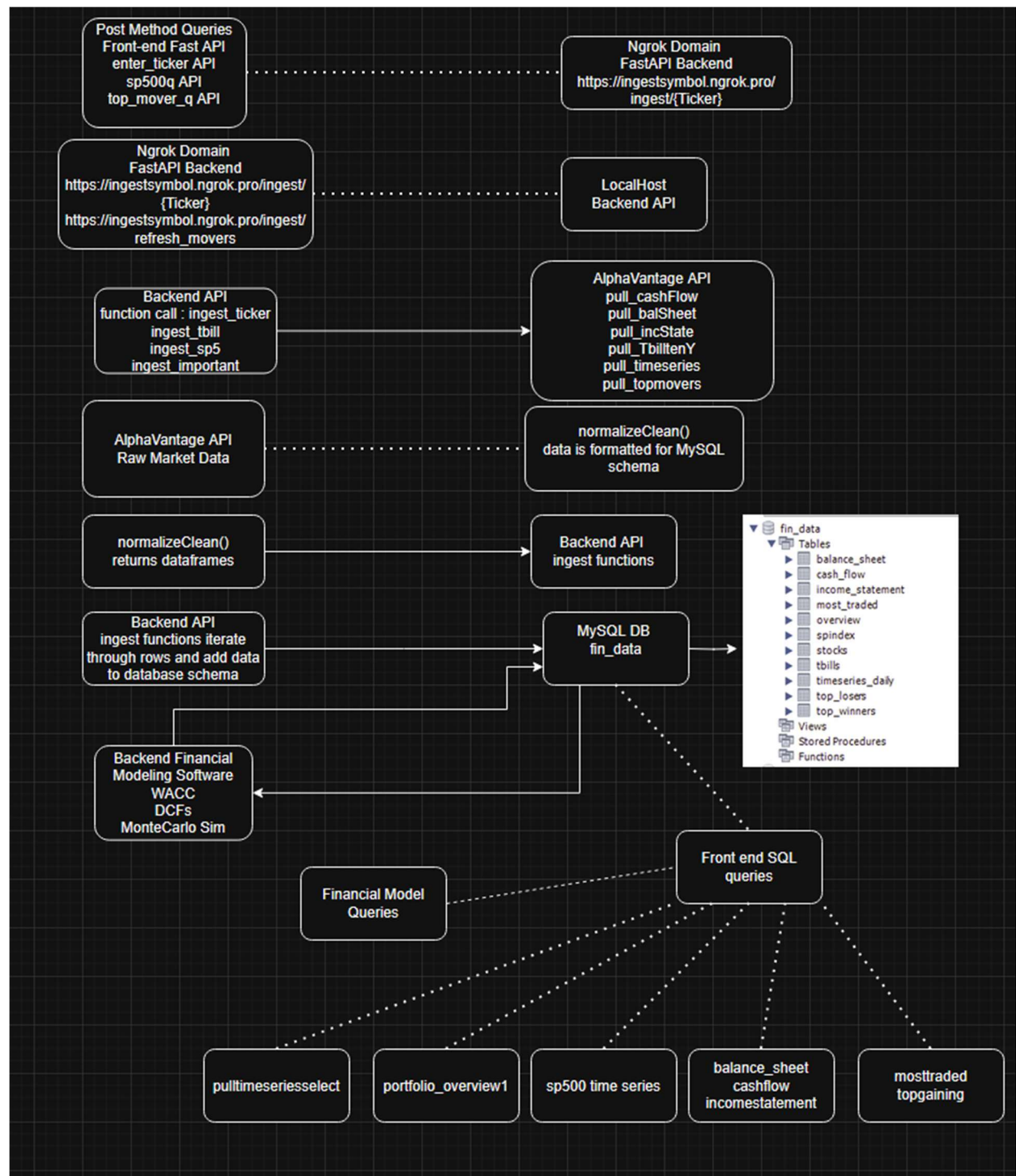
Frontend

- My user interface acts as a data-visualization tool for equity instruments. On the first page we have a portfolio that our users can build by entering a ticker representing an equity instrument. By entering a ticker, the user will trigger an API POST method calling data retrieval, formatting, and database commit functions. When a successful API POST is made the frontend will execute DB queries for the corresponding ticker retrieving the data and populating the table. In addition to the portfolio, we have two graphs which can compare the tickers price history in the market to the SP500's price history. Next to the graphs we have a snapshot of the most traded and most gaining stocks. This gives us insight into where investments are being placed and which stocks are rising the fastest. If our users want better insight into an equity instrument, then by double clicking on a row in the portfolio table the user is taken to page 2.
- On page 2 specific details are displayed following the click on the equity instrument. The user can see the last closing price and the performance in the market at that time. By selecting a financial statement for the stock the user can see the statements history by quarter. If a user has questions about what a metric means or how to conduct further analysis of the stock a LLM component provided by retool exists to answer questions.

Backend

- The backend software supports the frontend by creating a database, formatting the data, and adding the data to the database. Once an API request is received the ingestion function for equity instruments is received. We then retrieve our financial data from Alpha Vantage API and normalize the data by using Panda's modules. The data is then committed to the database. To host our API we use a custom Ngrok URL and then run the application over the uvicorn module making the backend accessible from Retool. Our database is also made accessible by forwarding our localhost:3306 service to a TCP Ngrok domain.

Context Diagram



System Functions

Main.py

```
from database import initialize_database, Base, engine # Optional: one-time DB
setup
from pyngrok import ngrok
import uvicorn, subprocess

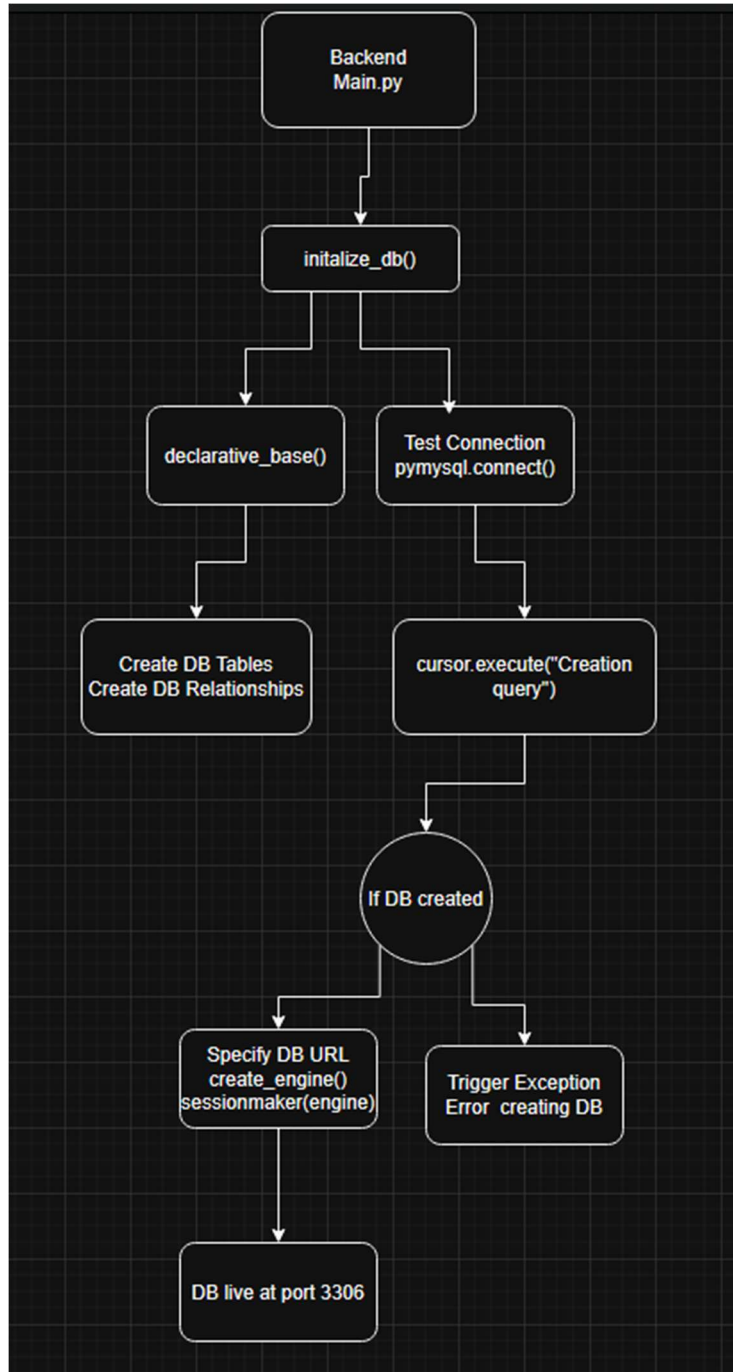
ngrok.set_auth_token("2aPOIu0sWqUH5ijSYf5xr2NFTzD_4Rmhn8BNNkpD4CzGCePGG")

def main():
    #First, we initialize the database, mapping the classes to the database
    schema
    print("Initializing database...")
    initialize_database()
    # Start a ngrok tunnel to the local FastAPI app
    public_url =
ngrok.connect(8000, proto="http", domain="ingestsymbol.ngrok.pro")
    # Run FastAPI app from data_injest.py
    print(f"App is live at: {public_url}")
    uvicorn.run("db_commit:app", host="0.0.0.0", port=8000)

if __name__ == "__main__":
    main()
```

- From the start of the program in main.py we call our database initialization function to create the database. Which will be explained in depth.
 - We then host the database over a public facing TCP connection with ngrok by entering the following command in the terminal which will forward traffic from our localhost service running on 3306 to the Ngrok domain
 - (.venv) PS C:\Users\zuck1\PycharmProjects\capstone_backend> ngrok tcp --remote-addr=5.tcp.ngrok.io:23276 3306
- Next our backend is exposed to the public facing internet through a custom Ngrok domain > <https://ingestsymbol.ngrok.pro/>
- Uvicorn allows our Fast API functions to be access by the frontend software made accessible through the above domain

Functional Decomposition Diagram



System Functions

Database Creation and Database Schema

database.py

- We create a MySQL database using the following modules
 - SQLAlchemy
 - SQLAlchemy.ext.declarative
 - SQLAlchemy.orm
 - We use these functions to conduct Object Relational Mapping in Python.
 - Dotenv
 - We use this module to create an environment file which contains the values of the user, password, host, port, and db name which are then set to variables needed to form a connection to the db
 - Pymysql
 - Python module used to connect to the MySQL database
 - Os
 - Module used for basic operating system commands
 - Ngrok
 - To make the database accessible to the front end we use ngrok's tunneling service to host the database over TCP connection on the public facing internet. All traffic from this TCP address is forwarded to the hosting machines localhost at port 3306

- Database Creation

- `Base = declarative_base()`
 - Declarative base function allows for automatic mapping and creation of tables by defining the table's attributes.

```
class Stock(Base):
    __tablename__ = 'stocks'
    ticker = Column(String(20), primary_key=True)

    overview = relationship("Overview", back_populates="stock",
                           cascade="all, delete-orphan")
    balance_sheet = relationship("BalanceSheet",
                                back_populates="stock", cascade="all, delete-orphan")
    income_statement = relationship("IncomeStatement",
                                    back_populates="stock", cascade="all, delete-orphan")
    cash_flow = relationship("CashFlow", back_populates="stock",
```

```

cascade="all, delete-orphan")
    spindex = relationship("SPindex", back_populates="stock",
cascade="all, delete-orphan")
    timeseries_daily = relationship("TimeSeriesDaily",
back_populates="stock", cascade="all, delete-orphan")
    tbills = relationship("TbillData", back_populates="stock",
cascade="all, delete-orphan")

```

- We define the parent table for equity instruments by declaring the primary key as the ticker or symbol for the instrument
- Relationships between the child tables are then set as the ticker back populates into the parent table. The primary key for the child tables is an integer value which increments as data flows into the table. A foreign key is used to tie the relationship back to the parent table Stock

- .ENV

- This file is used to specify the DB's environment variables. We leave the .env in the same directory as the database generator file so that we can access the file to set the variables with the code below.

```

○ DB_USER = os.getenv("FIN_DB_USER", "your_user")
  DB_PASSWORD = os.getenv("FIN_DB_PASSWORD", "your_password")
  DB_HOST = os.getenv("FIN_DB_HOST", "localhost")
  DB_PORT = os.getenv("FIN_DB_PORT", "3306")
  DB_NAME = os.getenv("FIN_DB_NAME", "fin_data")

```

- Pymysql

- We then use the env variables which have been set to test the connection to the DB

```

• try:
    connection = pymysql.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        port=int(DB_PORT),
        charset='utf8mb4',
        autocommit=True
    )
    with connection.cursor() as cursor:
        cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}
        CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;")
    connection.close()
    print(f"Verified or created MySQL database: {DB_NAME}")
except Exception as e:
    print(f"Error creating database: {e}")
    raise

```

- DB creation

```

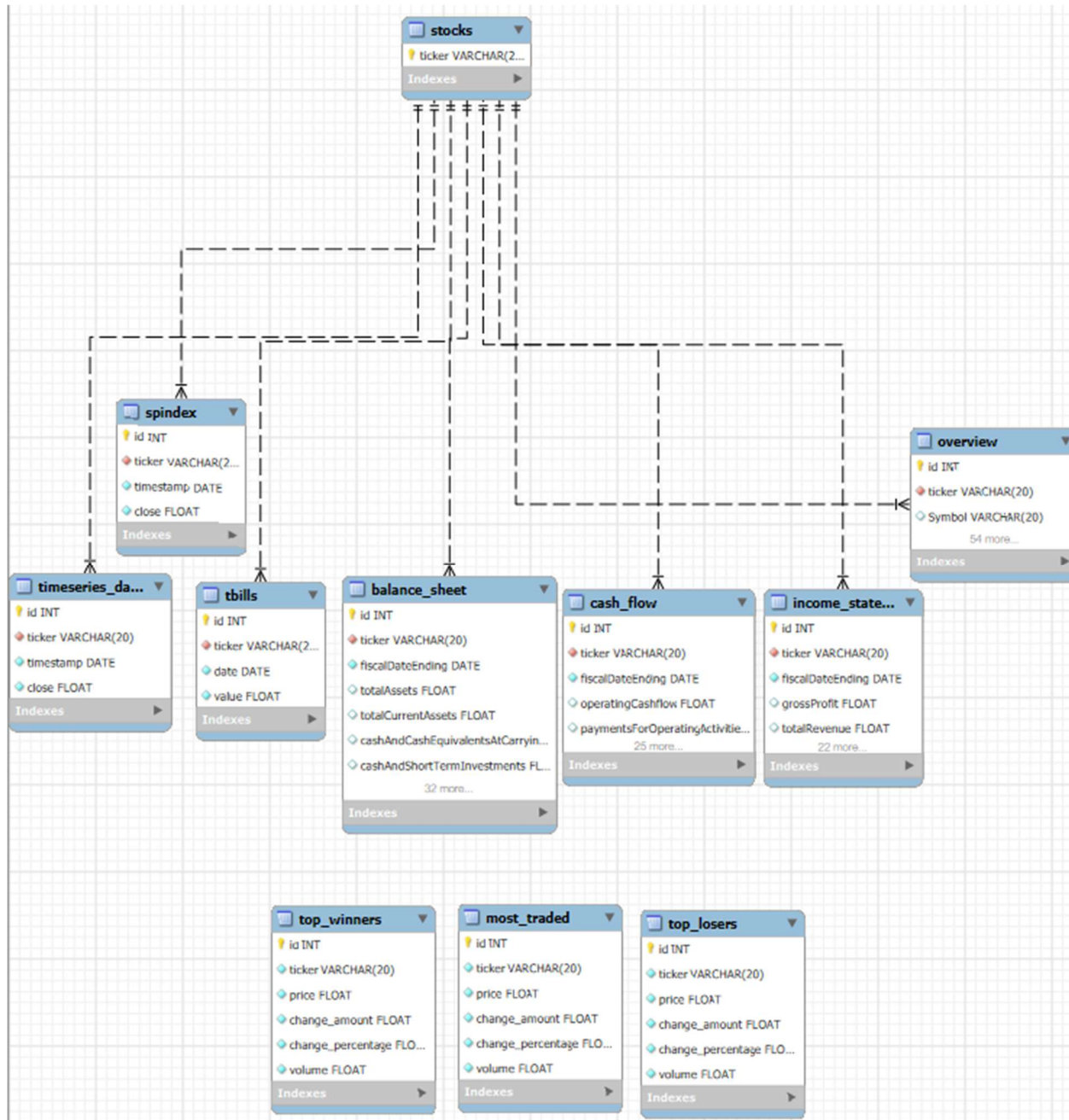
• DATABASE_URL =
  f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}
  ?charset=utf8mb4"

```

```
engine = create_engine(DATABASE_URL, echo=False)
Session = sessionmaker(bind=engine)
```

- We define the URL for the database using the environment variables and setting the character type
- The engine is then created and binds to the session maker to create an accessible session to add, commit, and delete data from the DB using Python.

ERD Diagram



SQL Queries

The below queries are triggered by user interaction with the frontend service triggering a post request to the backend database.

- Portfolio Overview
 - `SELECT * FROM fin_data.overview WHERE ticker IN ({{ variable4.value }});`
 - `SELECT * FROM fin_data.overview WHERE ticker IN ({{ localStorage.values.portfolio }}`
- Pull Historical Prices
 - `SELECT timestamp,close FROM fin_data.spindex`
- Pull Time Series Data
 - `SELECT * FROM fin_data.timeseries_daily WHERE ticker = ({{ variable4.value }}`
- Financial Statements
 - Balance Sheet
 - `SELECT * FROM fin_data.balance_sheet WHERE ticker = ({{ variable4.value }}`
 - Income Statement
 - `SELECT * FROM fin_data.income_statement WHERE ticker IN ({{ variable4.value }}`
 - Cash Flow Statement
 - `SELECT * FROM fin_data.cash_flow WHERE ticker IN ({{ variable4.value }}`
- Top Gaining Companies
 - `SELECT * FROM fin_data.top_winners`
- Most Traded Currently
 - `SELECT * FROM fin_data.most_traded`

Data Commit

db_commit.py

- Contains Fast API logic for backend endpoint.
 - Modules
 - Fastapi , fastapi.middleware.cors
 - data_ingest
 - functions from data ingestion python file
- Once a request is received at the Fast API domain the application triggers corresponding functions when POST requests are sent from the frontend.

```
• app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"], # or specify your Retool domain  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

- We add middle ware parameters to allow the post method to reach our application from all origins and all methods.
 - In the future allowing all origins, methods, and headers poses a security risk but for testing purposes we will leave it as is.

```
• @app.get("/")  
def root():
```

- Diagnostic response for GET method to notify app is running

```
• @app.post("/ingest/refresh_movers")  
def ingest_refresh_movers():
```

- Calls the data ingestion function to pull, format, and store market data on the most active stocks.

```
• @app.post("/ingest/{ticker}")  
def ingest_ticker(ticker: str):
```

- This function dynamically calls ingestion methods based on the ticker requested by the client. It is dynamic in that conditional logic tests for instrument type, ie; SPY Index, T-Bills, or Equity. The corresponding function is then triggered which will pull, format, and store the data. If the ingestion method fails then an HTTP exception error code is raised, and the error message is returned to the frontend

Data Ingest

db_injest.py

- This python file imports the previously written files database.py and AV_API_pull.py allowing us to use the database session connection and pull data from Alpha Vantage. It serves as the bridge between the AV API and the MySQL Database

```
• def ingest_sp5(sp: str):  
• def ingest_tbill(ticker: str):  
• def ingest_stock_data(ticker: str):
```

- These functions have the same structure in the way they iterate through incoming dataframes and add the dataframes to the MySQL database.

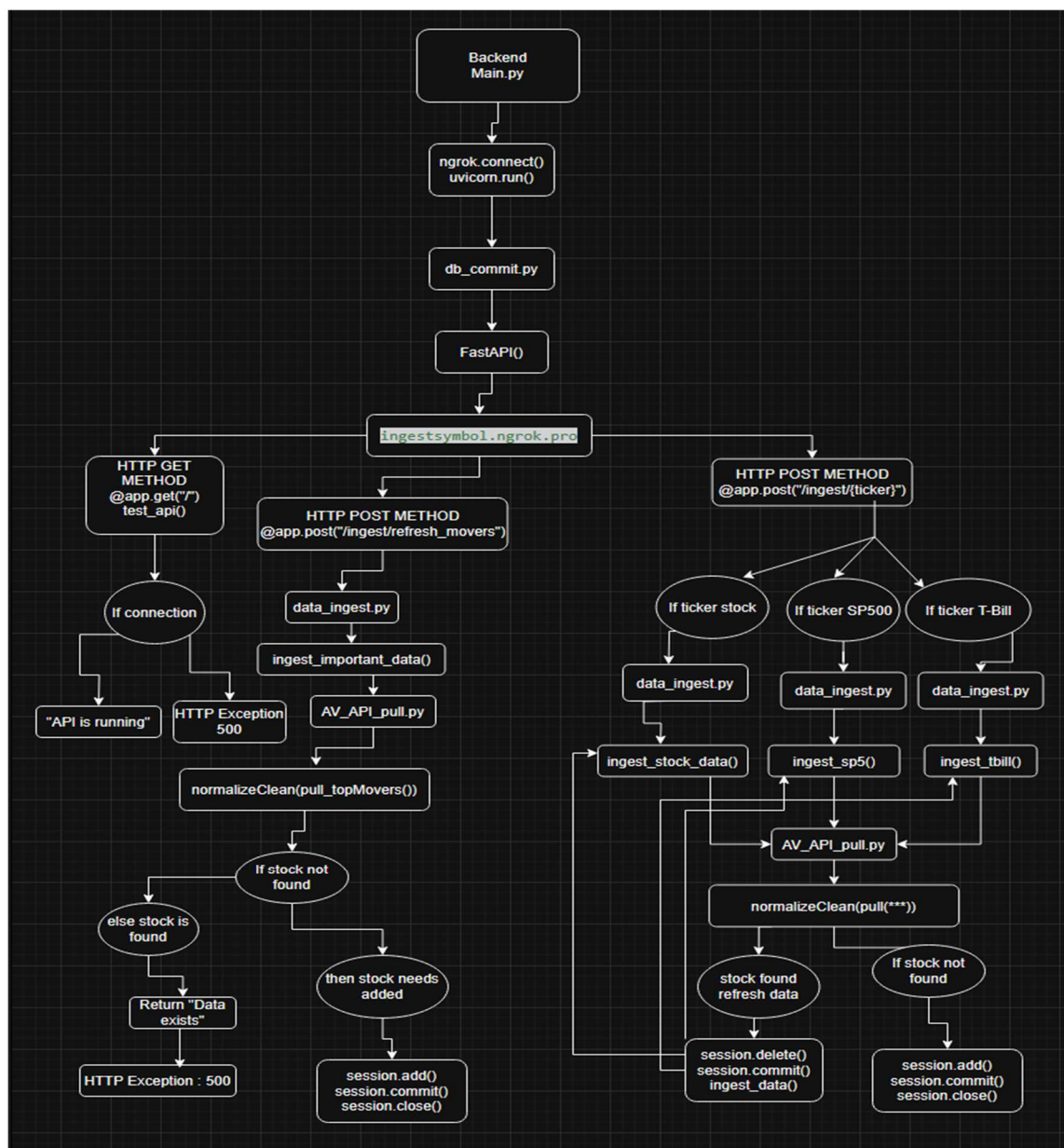
```
• stock = session.query(Stock).filter_by(ticker=ticker).first()  
• overview = normalizeClean(pull_Overview(ticker))  
  balance_sheet = normalizeClean(pull_balSheet(ticker))  
  income_statement = normalizeClean(pull_incState(ticker))  
  cash_flow = normalizeClean(pull_cashFlow(ticker))  
  time_series = normalizeClean(pull_timeSeriesPriceDaily(ticker))  
•  
• if not stock and overview is not None:  
  print("Stock not found updating entry...")  
  print(stock)  
  stock = Stock(ticker=ticker)  
  session.add(stock)  
  session.commit()  
  
  if time_series is not None:  
    for _, row in time_series.iterrows():  
      time_obj = TimeSeriesDaily(ticker=ticker, **row)  
      session.add(time_obj)  
  
  # ---- Insert Overview ----  
  if overview is not None:  
    row = overview.iloc[0].to_dict()  
    overview_obj = Overview(ticker=ticker, **row)  
    session.add(overview_obj)
```

- First a session object is declared
 - The ticker is then added to the parent table for the database.
- Data Normalization function is called
 - Function described in AV API function description below.
- Conditional logic checks for the existence of the stock ticker and for empty dataframes.
 - If empty dataframes

- We then iterate through the dataframe by index and row adding all row contents to the corresponding database table

```
for _, row in time_series.iterrows():
    time_obj = TimeSeriesDaily(ticker=ticker, **row)
    session.add(time_obj)
```

Functional Decomposition Diagram



AVI_API_Pull

AV_API_pull.py

- This python file interfaces with AlphaVantage API to retrieve current stock market data.
 - Modules
 - Requests
 - Requests is used to access AlphaVantage API and return a Json or csv object.
 - NumPy
 - Used to change None values into NaN to format null data correctly for MySQL
 - Pandas
 - Used to format Json data into normalized data frames ready for insertion to MySQL
 - Stringlo
 - Used to treat buffer objects returned by pull_TimeSeriesPriceDaily and pull_SP500 as a txt file which can be converted to a csv
 - AlphaVantage API
 - Leading provider of premium consumer stock market data as close to real-time as possible per FINRA/SEC regulation.
 - Accessed through API Key

- Pull Functions

- `def pull_Overview(symbol):`

- Pulls overview data on stock symbol entered by user
 - Overview data includes standard metrics for first pass analysis of a stock such as Buy/Sell ratings, PE Ratio, Beta, Market Capitalization, Outstanding Shares, Shares Float, and profit margin.

- Returns Json time series data

- `def pull_balSheet(symbol):`

- `def pull_incState(symbol):`

- `def pull_cashFlow(symbol):`

- Pulls financial statements related to stock symbol entered by user

- Returns Json time series data

- `def pull_TbilltenY(symbol):`

- Pulls 10y T-Bill historical data to be used for risk-free rate calculation

- Returns Json object

```
• def pull_timeSeriesPriceDaily(symbol)
• def pull_SP500(symbol):
```

- Time-Series data related to historical prices of SP500 Index and related symbol.
- Returns Panda's Df

```
• def pull_topMovers():
```

- Pulls most traded stocks, top gaining stocks, and top losing stocks.
 - Ticker, Volume, Price, Change Amount, Change Percentage
 - Returns Json object

- Data Normalization

```
• def normalizeClean(data):
```

- Takes in Json and Panda's dataframes and tests for certain patterns within the data returned by the pull functions. For example each financial statement has slightly different column headers which creates a pattern that can separate its normalization logic from the other conditional normalization logic meant for the specific statement.

```
• if ("data" in data):
```

- Identifies 'data' key within 10y T-Bill data utilizing built in Panda's normalization function
- Sets 'Date' column to DateTime obj and sets 'value' column to float data type

```
• elif ("top_gainers" in data):
```

- Normalizes top moving stock data into pandas data frame
- Sets data type for respective columns using custom data type setter function

```
○ def set_datatype(df):
```

- Ticker column data returned to Df as strings
- Change Percentage column data returned to Df as float
- All other column data is turned into float data type

- Returns three data frames top_gainers, top_losers, most_actively_traded

```
• elif (isinstance(data, pd.DataFrame) and "close" in data.columns):
```

- Tests for existing TimeSeries dataframe and 'close' column
- Separates 'close' column from all other data

```
• elif ("Description" in data and "Beta" in data):
```

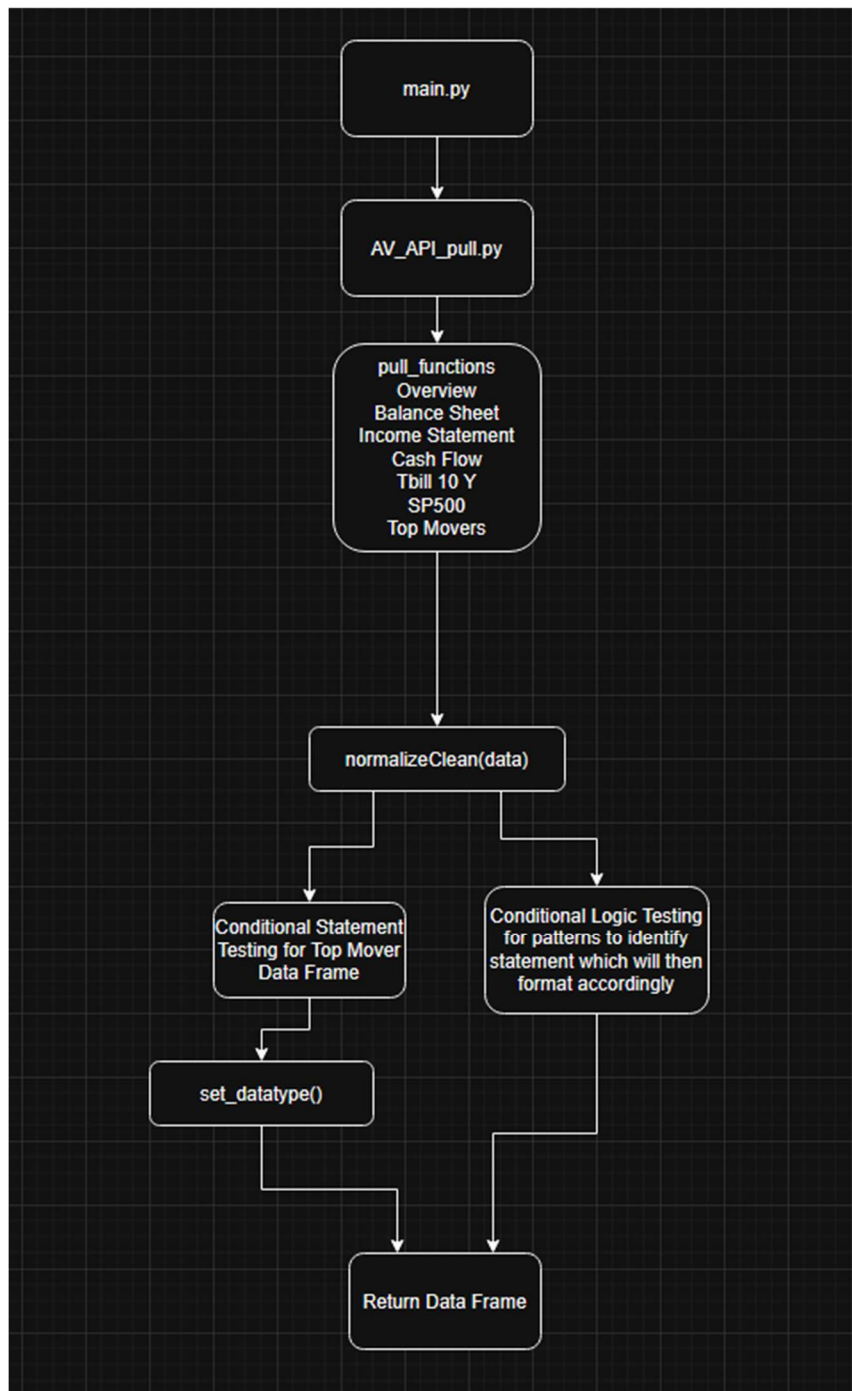
- Tests for Overview data

- Normalizes data and replaces null values with acceptable data for MySQL
- Renames columns with a leading '_' to avoid issues with naming conventions in Python.

```
• elif ("quarterlyReports" in data):
```

- Tests financial statements for quarterlyReports key value and normalizes the data on this key in the Json data
- Replaces NaN values with None values which are accepted by MySQL

Functional Decomposition Diagram

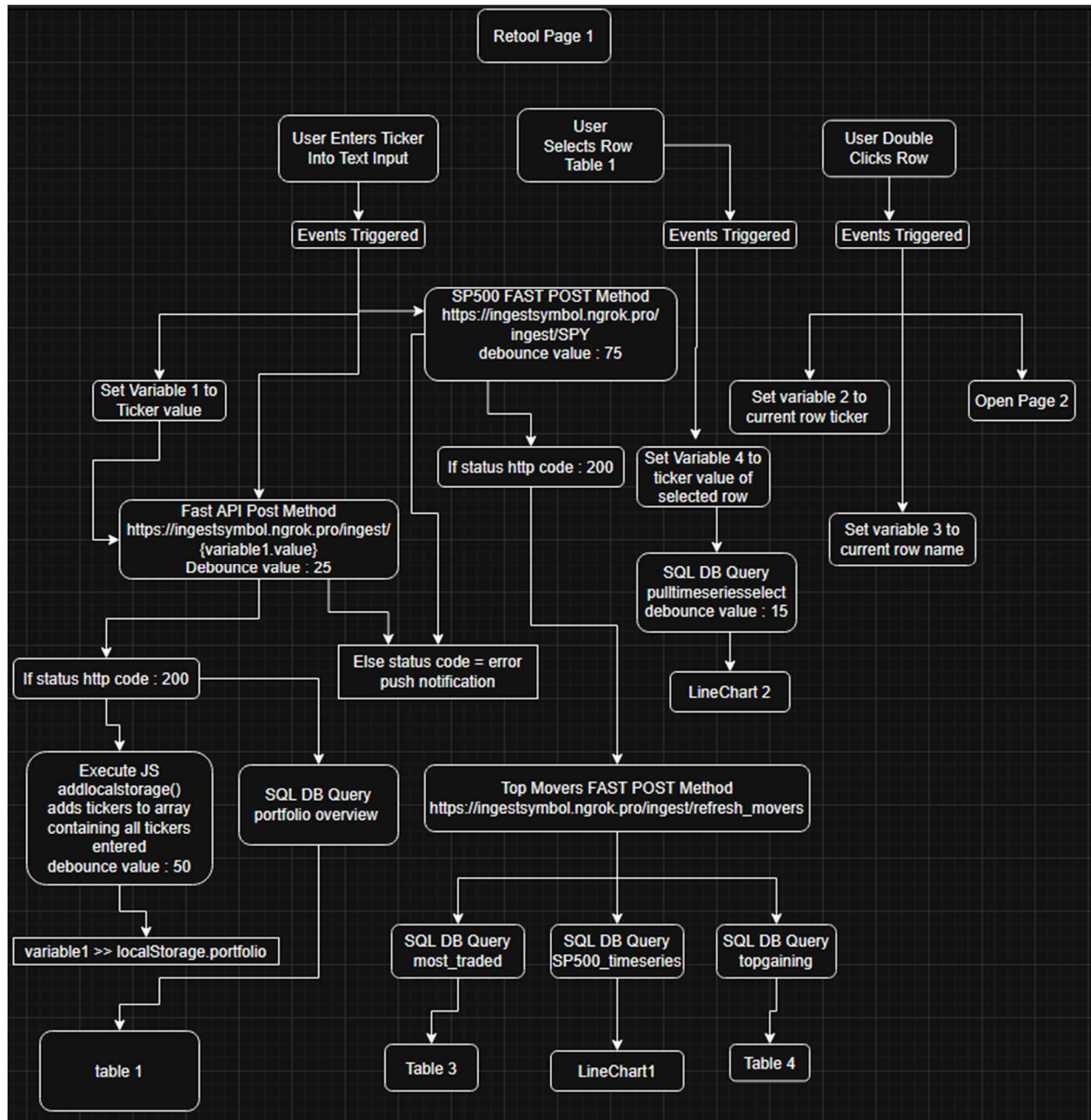


Event List

The responses to the events are shown as a cascading effect in the list into other events. An unsuccessful event returns an error code			
Event 1	Event 2	Event 3	Event 4
Set Var 1	Post API Calls Query : enter_ticker	Post API Calls Query : top_mover_q	SQL Query DB : SP500_timeseries
	Post API Calls Query : SP500_timeseries	SQL DB Query : Portfolio Overview	SQL Query DB : most_traded
	JS Query 7 adds ticker to local storage		SQL Query DB : topgaining
Query DB for timeseries data on ticker			
Load Page 2			
Set Var 2			
Set Var 3			
Set Var 4			
Event 1	Event 2		
JS Query : Pullfindata	SQL Query DB : Executes dynamically according to fin statement		

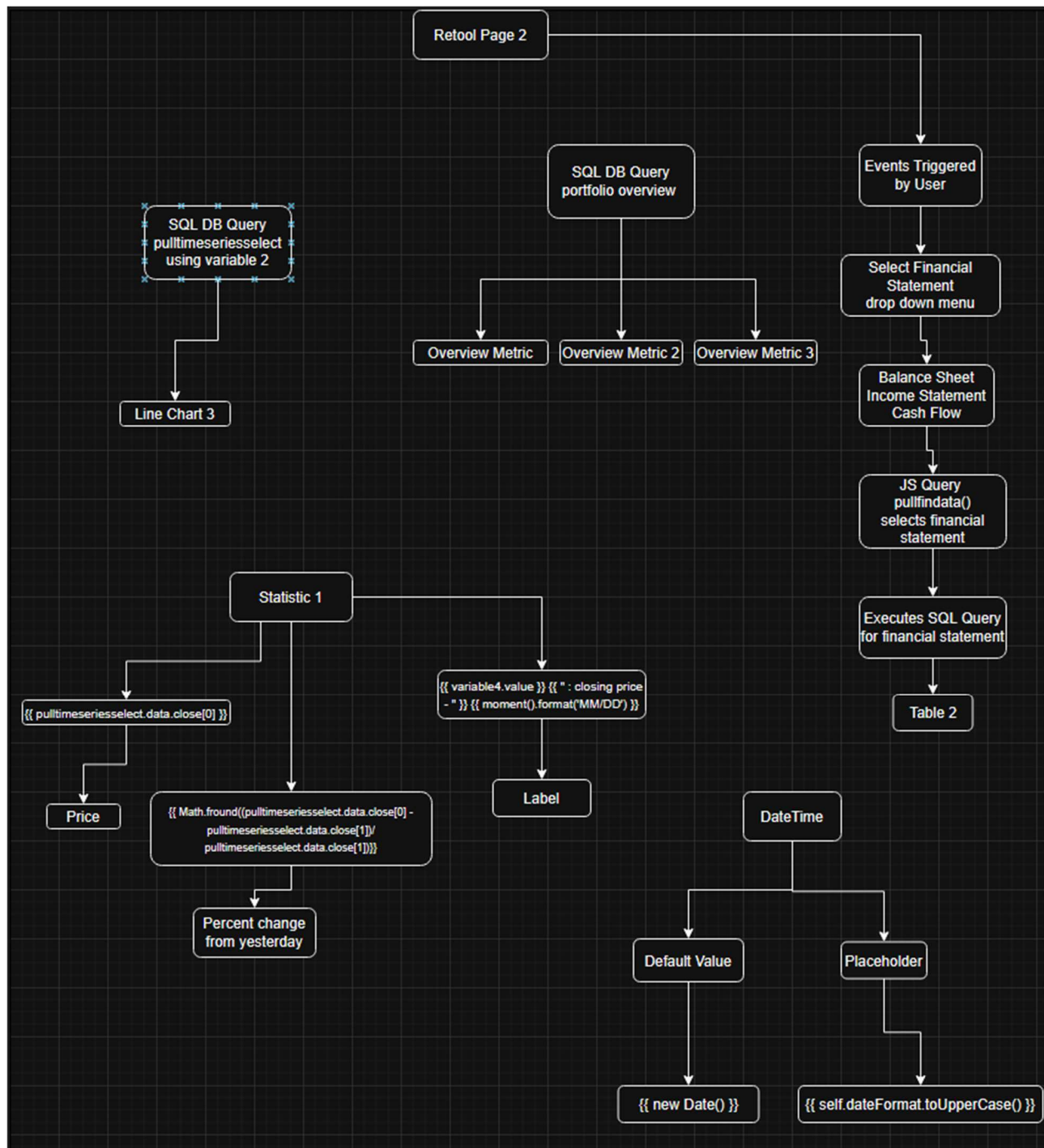
Event Diagrams

Page 1



Event Diagrams

Page 2



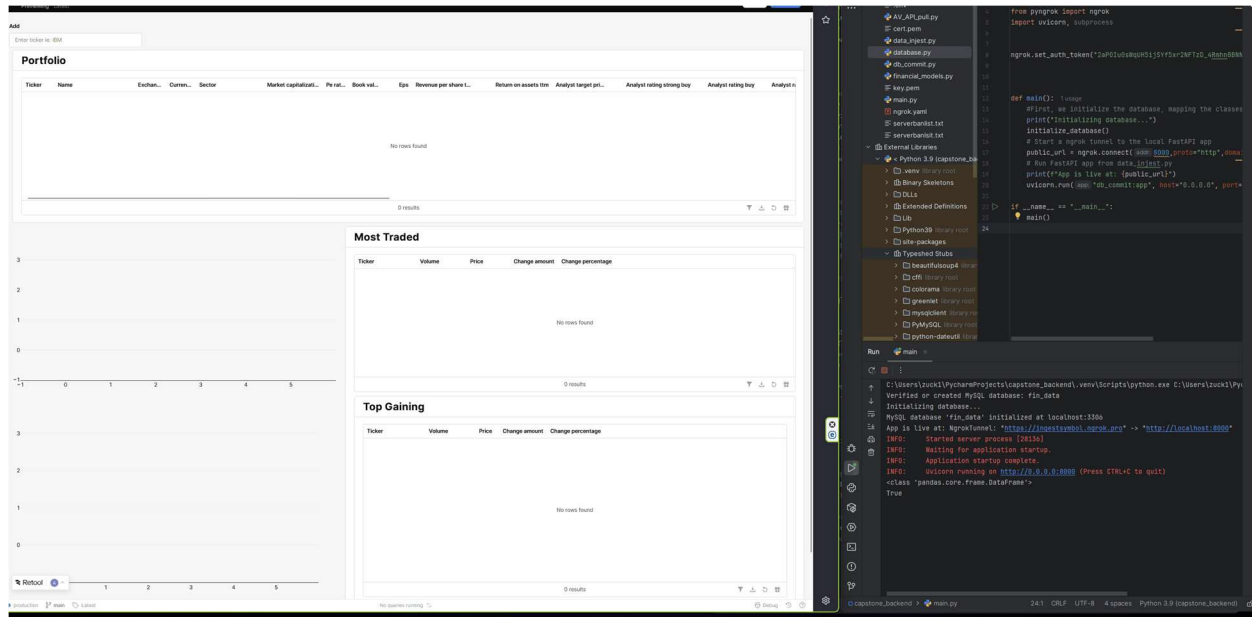
Normalized Database Design

- I found it less than ideal for financial data which presents itself in a standardized format to be normalized to 3NF. Each column in the financial statements whether they have null values for one ticker needs to be present in the case that an atypical ticker is entered where the column is populated. We therefore save space and time by pooling the data into one standardize format with null values.
 - For example IBM has years where accumulatedDepreciationAmorization is recorded as a null value and then years where this expense is recorded with a value that adds weight to the assets and liabilities calculations.
 - NVDA on the other hand does not record this expense at all on there balance sheet
- I achieved 2NF in the database by creating a primary key which increments for every entry and a foreign key designated at the ticker. This makes each line item a unique entry.

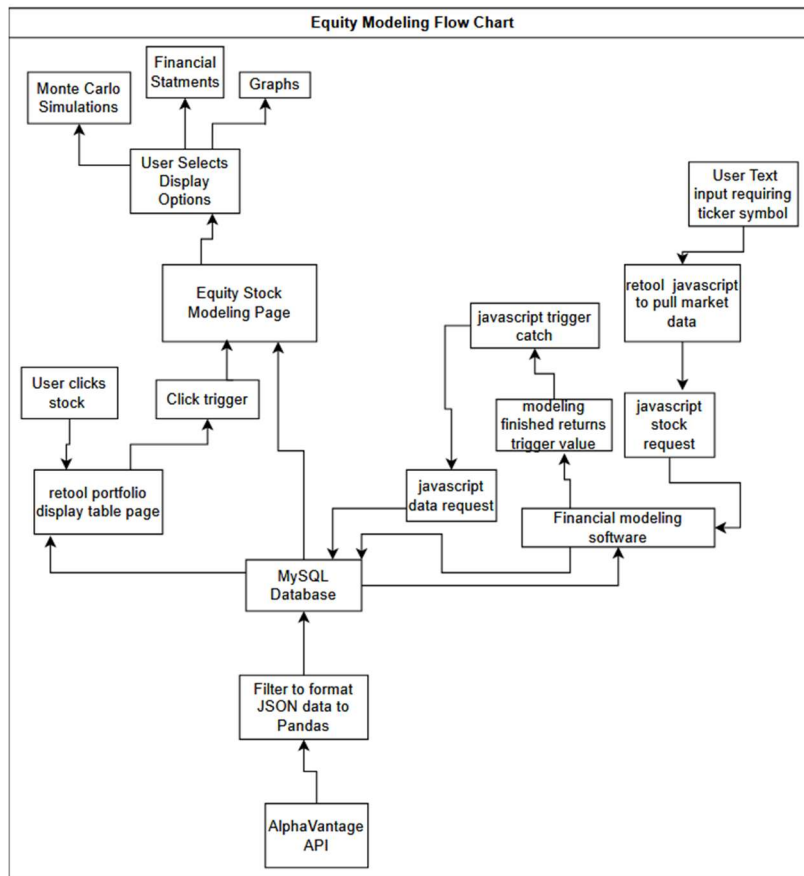
Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Contents:																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
-------------	--	--------------	--	-------	--	----------------	--	---------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

User Interface Design

- The below gif shows the user interface design in action as the user operates the software.



Flow Chart



Test Report

```
> • Python 3.9 (capstone_backend) > C:\Users\jack\Pychar 113  
    > venv \Library\bin 114  
    > db Binary Skeletons 115  
    > @DLS 116  
  
Run main x  
None  
Starting modeling software for : IBM  
INFO: 35.90.103.134:- "POST /ingest/IBM HTTP/1.1" 200 OK  
C:\Users\juckl\P\pycharmProjects\capstone_backend\AV_API_pull.py:118: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, explicitly call '.result'.  
df.replace("none", np.nan, inplace=True)  
C:\Users\juckl\P\pycharmProjects\capstone_backend\AV_API_pull.py:118: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, explicitly call '.result'.  
df.replace("none", np.nan, inplace=True)  
C:\Users\juckl\P\pycharmProjects\capstone_backend\AV_API_pull.py:118: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, explicitly call '.result'.  
df.replace("none", np.nan, inplace=True)  
Stock not found updating entry...  
None  
(pymysql.err.ProgrammingError) nan can not be used with MySQL  
SQL: INSERT INTO overview (ticker, Symbol, AssetType, Name, Description, CIK, Exchange, Currency, Country, Sector, Industry, Address, OfficialSite, FiscalYearEnd, LatestQuarter, MarketCapitaliz  
ionYears) VALUES ('RAYA', Symbol='RAYA', AssetType='Common Stock', Name='Erayak Power Solution Group Inc.', Common Stock', Description:'Erayak Power Solution Group Inc., engages in the research an  
(Background on this error at: https://sqlalche.me/e/20/c4ao)  
INFO: 35.90.103.132:- "POST /ingest/RAYA HTTP/1.1" 500 Internal Server Error
```

- Input Ticker : RAYA
 - After inputting RAYA as a ticker there was a data discrepancy in some fields that were not populated.
 - To investigate I took the Alpha Vantage API interface to Jupyter Notebook where I could analyze the data in a notebook style.
 - Using display() to show the dataframe in its formatted state I was able to identify that the issue was my database schema will not accept NaN or None values.
 - To fix this I set the value to zero to keep the data consistent.

The screenshot displays a Jupyter Notebook environment with a Python function and its output in a terminal window.

Jupyter Notebook Code:

```
def get_data():
    # Connect to the database
    db = psycopg2.connect(database='postgres', user='postgres')
    # Create a cursor
    cursor = db.cursor()
    # Execute the query
    cursor.execute("SELECT * FROM public.get_data()")
    # Fetch all the rows
    rows = cursor.fetchall()
    # Close the connection
    db.close()
    # Return the data
    return rows

# Call the function
data = get_data()

# Print the data
print(data)
```

Terminal Output:

```
[21 rows x 28 columns]
Starting modeling software for : RAYA
INFO: 35.90.103.132:0 - "POST /ingest/RAYA HTTP/1.1" 200 OK
```

References

1. *Financial Modeling Guide: Excel training tutorial*. Wall Street Prep. (2025, April 7). <https://www.wallstreetprep.com/knowledge/financial-modeling/>
2. VirtualCoin CISSP, P. (2023, November 16). *Tutorial python - mysql database using sqlalchemy orm*. TechExpert. <https://techexpert.tips/python/python-mysql-database-sqlalchemy-orm/>
3. Brumm], map[name:Ben. “Database Normalization: A Step-by-Step-Guide with Examples.” *Database Star: Home*, Database Star: Home, 30 Sept. 2017, www.databasestar.com/database-normalization/.
4. Hilpisch, Y. (2018). *Python for Finance, 2nd edition*. O’Reilly Media, Inc.
5. Pignataro, P. (2022). *Financial modeling and valuation*.
6. Vantage, A. (n.d.). *Free stock apis in JSON & Excel: Alpha vantage*. Free Stock APIs in JSON & Excel | Alpha Vantage. <https://www.alphavantage.co/>